

Rapport – Application Bancaire Mobile

1. Introduction

L'objectif du projet est de développer une **application bancaire mobile** permettant aux utilisateurs :

- de pouvoir s'inscrire et de se connecter de manière sécurisée,
- de consulter leur solde,
- de faire des transferts d'argent entre utilisateurs
- d'utiliser un QR Code permettant de simplifier les transferts,
- de recharger leur compte via stripe

L'application se compose de deux parties principales :

- Frontend mobile avec Flutter,
- Backend sécurisé avec Laravel et API REST.

2. Architecture du projet

Frontend

- Gestion de l'interface utilisateur et des interactions.
- Scanner et générateur de QR Code.
- Appels à l'API pour récupérer le solde, effectuer des transferts et gérer les rechargements.
- Gestion sécurisée du token JWT pour authentification.

Backend

- API sécurisée avec JWT afin de protéger les routes sensibles
- Gestion des transactions financières et des utilisateurs
- Intégration avec Stripe pour recharger les comptes
- Gérer la base de données pour les utilisateurs, transactions et produits

3. Authentification

- Possibilité pour les utilisateurs de s'inscrire avec nom, mail et mot de passe
- Le mots de passe est haché et sécurisé
- Au moment de la connexion, un token JWT est généré afin de sécuriser les communications
- L'API vérifie les identifiants et renvoie le token et les informations de l'utilisateur

4. Gestion des transferts

- Les utilisateurs peuvent transférer de l'argent à d'autres utilisateurs identifiés par un ID unique.
- La sécurité est renforcé par l'utilisation de transactions dans la base de donnée pour éviter les incohérences.
- Le solde de l'émetteur et du receveur est mis à jour en temps réel
- Le transfert est déclenché manuellement ou via la lecture d'un QR Code

5. QR Code

- Chaque utilisateur dispose de son QR code pour récupérer l'ID du destinataire
- Le QR code permet de faciliter les transferts en évitant la saisie des identifiants
- L'application mobile peut scanner un QR Code pour récupérer l'ID destinataire.

6. Rechargement de compte via Stripe

- Les utilisateurs peuvent recharger leur compte via Stripe
- Une session de paiement est créée côté serveur
- Après validation du rechargement le solde de l'utilisateur est mis à jour

-Un webhook Stripe permet de mettre à jour automatiquement le solde après rechargement.

7. Consultation du solde

-Les utilisateurs peuvent consulter leur solde dans l'application mobile

8. Routes API et sécurité

-Routes publiques : inscription, connexion et consultation des produits.

-Routes protégées par JWT : consultation du solde, transfert d'argent, création de sessions Stripe.

-Les routes Stripe pour le rechargement et les webhooks ne nécessitent pas de JWT pour garantir la compatibilité avec Stripe.

9. Configuration

-La configuration de l'application inclut :

-les clés Stripe pour test et production,

-le secret JWT,

-les informations de base de données SQLite,

-l'URL de l'API backend accessible depuis le mobile.

10. Difficultés rencontrées

-Gestion des tokens et sécurisation des appels API.

-Format des QR Codes pour inclure les informations nécessaires.

-Gestion des erreurs côté serveur et côté mobile pour les transferts et rechargements.

-Synchronisation du solde après rechargement via Stripe.

11. Solutions et améliorations

- Mise en place d'un QR Code structuré avec l'ID utilisateur.
- Transferts sécurisés avec transaction SQL pour éviter les pertes de données.
- Utilisation de messages clairs et SnackBars dans l'application mobile pour informer l'utilisateur.
- Redirection automatique vers Stripe pour le rechargement.

12. Conclusion

- L'application bancaire mobile propose un systeme complet:
- L'authentification est sécurisée,
- Consultation de solde,
- Transfert via QR Code ou saisie de l'ID
- Recharger son compte via stripe

Enfin elle représente un projet de wallet mobile, avec des combinaisons de sécurité, ergonomie et intégrations de service financiers

Indications pour le test de l'application bancaire

Utilisation de ngrok :

Pour que l'application Flutter puisse communiquer avec le backend Laravel en local, ngrok a été utilisé pour exposer l'API sur une URL publique temporaire. Cela permet de tester les fonctionnalités réseau (transfert, rechargement via Stripe) sans déployer le backend sur un serveur distant.

Test du QR Code pour les transferts :

Lors des tests, il ne faut pas entrer directement l'ID du destinataire dans le champ prévu à cet effet. Après avoir rempli le montant et ouvert l'écran de transfert, il suffit de cliquer sur « Scanner QR » pour récupérer automatiquement l'ID du destinataire via le QR Code généré par l'application.

Test du rechargement via Stripe :

Après avoir cliqué sur « Recharger mon compte » dans l'application :

Une session Stripe Checkout est créée et un lien de paiement est généré.

Pour simuler un paiement réussi dans l'environnement de test, ouvrir un PowerShell ou un terminal et exécuter la commande :

```
stripe trigger checkout.session.completed
```

Cette commande envoie un événement webhook à l'API, déclenchant la mise à jour du solde utilisateur comme si le paiement avait été effectué avec succès.

